

Enterprise Database Infrastructure Administration Lab

Design, Deployment, Security Hardening, Backup, Replication and
Monitoring of PostgreSQL Infrastructure on Linux

David Gottlieb SITTI
<https://www.linkedin.com/in/davidgottliebsitti/>

SUMMARY

POSTGRESQL REPLICATION & HIGH AVAILABILITY FOUNDATIONS	2
--	---

POSTGRESQL REPLICATION & HIGH AVAILABILITY FOUNDATIONS

I. Objective

The purpose of this deliverable is to implement a highly available PostgreSQL environment by configuring Streaming Replication between :

- pg-primary (Primary Database Server)
- pg-replica (Standby Database Server)

At the end of this phase :

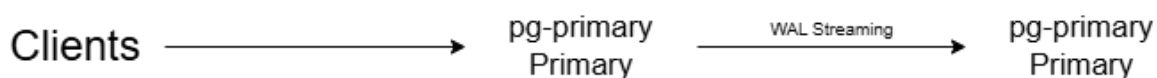
- Streaming replication operational
- Replica synchronized with primary
- Replication monitoring implemented
- Replication slot configured
- Failover procedures documented
- Recovery procedures tested
- Enterprise-grade HA foundation established

II. PostgreSQL Replication Architecture

1. Current Architecture

pg-primary : Single point of failure.

2. Target Architecture



III. Replication Concepts

Before touching the configuration, understand the components.

- **WAL**
 - Write-Ahead Log
 - Every database change is written to WAL before being committed.
- **Streaming Replication**
 - The primary continuously sends WAL records to the standby.
- **Standby Server**
 - Read-only copy of the primary database.
- **Replication Slot**
 - Prevents WAL files from being deleted before the replica receives them.

IV. Configure Primary Server

Perform on pg-primary.

Verify Current Settings

```
sudo -u postgres psql
SHOW wal_level;
```

```
You are now connected to database "postgres" as user "postgres".
postgres=# SHOW wal_level;
wal_level
-----
replica
(1 row)

postgres=#
```

SHOW max_wal_senders;

```
postgres=# SHOW max_wal_senders;
max_wal_senders
-----
10
(1 row)

postgres=#
```

SHOW archive_mode;

```
postgres=# SHOW archive_mode;
archive_mode
-----
on
(1 row)

postgres=#
```

V. Create Replication User

Create dedicated replication account.

CREATE ROLE replicator

WITH LOGIN

REPLICATION

PASSWORD 'ReplicationPassword2026!';

```
postgres=# CREATE ROLE replicator
postgres=# WITH LOGIN REPLICATION PASSWORD 'ReplicationPassword2026!';
CREATE ROLE
postgres=# \du

```

Role name	Attributes
app_user	
backup_operator	Replication
dba_admin	Superuser, Create role, Create DB
postgres	Superuser, Create role, Create DB, Replication, Bypass RLS
replicator	Replication

```
postgres=#
```

VI. Configure PostgreSQL Parameters

Edit :

sudo nano /etc/postgresql//main/postgresql.conf*

Locate and configure :

```
listen_addresses = '*'
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
wal_keep_size = 1024MB
hot_standby = on
```

Restart :

```
sudo systemctl restart postgresql
```

Verify :

```
sudo systemctl status postgresql
```

VII. Configure Replication Access

Edit :

```
sudo nano /etc/postgresql/*/main/pg_hba.conf
```

Add :

```
host replication replicator 192.168.10.21/32 scram-sha-256
```

Also ensure :

```
host all all 192.168.10.0/24 scram-sha-256
```

Restart :

```
sudo systemctl restart postgresql
```

VIII. Create Replication Slot

Connect :

```
sudo -u postgres psql
```

Create slot :

```
SELECT *
FROM pg_create_physical_replication_slot(
'replica_slot'
);
```

Verify :

```
SELECT *
FROM pg_replication_slots;
```

```
postgres=# SELECT *
FROM pg_replication_slots;
 slot_name | plugin | slot_type | datoid | database | temporary | active | active_pid | xmin | catalog_xmin | restart_lsn | confirmed_flush_lsn | wal_status | safe_wal_size | two_phase | conflicting
-----
 replica_slot |      | physical |        |          | f         | f      |             |      |              |             |                   |           |              |          |
(1 row)

postgres=#
```

IX. Prepare Replica Server

Move to pg-replica.

Stop PostgreSQL

```
sudo systemctl stop postgresql
```

```
sudo systemctl status postgresql
```

X. Remove Existing Cluster Data

Identify cluster location :

```
sudo -u postgres psql -c "SHOW data_directory;"
```

Remove contents :

```
sudo rm -rf /var/lib/postgresql/*/main/*
```

XI. Create Base Backup

This step clones the primary server.

On pg-replica :

```
sudo -u postgres pg_basebackup \
-h pg-primary \
-D /var/lib/postgresql/16/main \
-U replicator \
-P \
-R \
-X stream \
-S replica_slot
```

```
root@pg-replica:~#
root@pg-replica:~# sudo -u postgres pg_basebackup -h pg-primary -D /var/lib/postgresql/16/main -U replicator -P -R -X stream -S replica_slot
Password:
53884/53884 kB (100%), 1/1 tablespace
root@pg-replica:~# █
```

XII. Verify Replica Configuration

The -R option automatically creates :

```
standby.signal
```

and updates :

```
postgresql.auto.conf
```

Verify :

```
sudo ls -l \
/var/lib/postgresql/16/main/
```

Expected :

```
standby.signal
```

```

root@pg-replica:~# ls -l /var/lib/postgresql/16/main/
total 396
-rw----- 1 postgres postgres 225 juin 15 17:28 backup_label
-rw----- 1 postgres postgres 314396 juin 15 17:28 backup_manifest
drwx----- 9 postgres postgres 4096 juin 15 17:28 base
-rw----- 1 postgres postgres 60 juin 15 17:28 current_logfiles
drwx----- 2 postgres postgres 4096 juin 15 17:28 global
drwx----- 2 postgres postgres 4096 juin 15 17:28 log
drwx----- 2 postgres postgres 4096 juin 15 17:28 pg_commit_ts
drwx----- 2 postgres postgres 4096 juin 15 17:28 pg_dynshmem
drwx----- 4 postgres postgres 4096 juin 15 17:28 pg_logical
drwx----- 4 postgres postgres 4096 juin 15 17:28 pg_multixact
drwx----- 2 postgres postgres 4096 juin 15 17:28 pg_notify
drwx----- 2 postgres postgres 4096 juin 15 17:28 pg_replslot
drwx----- 2 postgres postgres 4096 juin 15 17:28 pg_serial
drwx----- 2 postgres postgres 4096 juin 15 17:28 pg_snapshots
drwx----- 2 postgres postgres 4096 juin 15 17:28 pg_stat
drwx----- 2 postgres postgres 4096 juin 15 17:28 pg_stat_tmp
drwx----- 2 postgres postgres 4096 juin 15 17:28 pg_subtrans
drwx----- 2 postgres postgres 4096 juin 15 17:28 pg_tblspc
drwx----- 2 postgres postgres 4096 juin 15 17:28 pg_twophase
-rw----- 1 postgres postgres 3 juin 15 17:28 PG_VERSION
drwx----- 3 postgres postgres 4096 juin 15 17:28 pg_wal
drwx----- 2 postgres postgres 4096 juin 15 17:28 pg_xact
-rw----- 1 postgres postgres 429 juin 15 17:28 postgresql.auto.conf
-rw----- 1 postgres postgres 0 juin 15 17:28 standby.signal
root@pg-replica:~#

```

Check :

```

sudo cat \
/var/lib/postgresql/16/main/postgresql.auto.conf

```

Expected :

primary_conninfo

```

root@pg-replica:~# cat /var/lib/postgresql/16/main/postgresql.auto.conf
# Do not edit this file manually!
# It will be overwritten by the ALTER SYSTEM command.
primary_conninfo = 'user=replicator password= channel_binding=prefer host='pg-primary' port=5432 sslmode=prefer sslcompression=0 sslcertmode=allow sslsni=1 ssl_min_pr
otocol_version=TLSv1.2 gssencmode=prefer krbsrvname=postgres gssdelegation=0 target_session_attrs=any load_balance_hosts=disable'
primary_slot_name = 'replica_slot'
root@pg-replica:~#

```

XIII. Start Replica

sudo systemctl start postgresql

sudo systemctl status postgresql

```

root@pg-replica:~# systemctl start postgresql
root@pg-replica:~# systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Mon 2026-06-15 16:33:25 UTC; 1h 2min ago
   Process: 11260 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 11260 (code=exited, status=0/SUCCESS)
   CPU: 1ms

juin 15 16:33:25 pg-replica systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
juin 15 16:33:25 pg-replica systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
root@pg-replica:~#
root@pg-replica:~#

```

XIV. Validate Standby Status

On pg-replica :

```

sudo -u postgres psql

```

Run :

```

SELECT pg_is_in_recovery();

```

Expected :

```
t
root@pg-replica:~# sudo -u postgres psql
psql (16.14 (Ubuntu 16.14-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# SELECT pg_is_in_recovery();
 pg_is_in_recovery 
-----
 t
(1 row)

postgres=#
```

Meaning : Replica operating is standby mode.

XV. Verify Replication from Primary

On pg-primary :

```
sudo -u postgres psql
```

Check :

```
SELECT
application_name,
client_addr,
state,
sync_state
FROM pg_stat_replication;
```

Expected :

```
pg-replica
streaming
```

```
postgres=# SELECT
application_name,
client_addr,
state,
sync_state
FROM pg_stat_replication;
 application_name | client_addr | state   | sync_state 
-----+-----+-----+-----
 16/main          | 192.168.10.21 | streaming | async
(1 row)

postgres=#
```

XVI. Replication Validation Test

1. On pg-primary

Connect :

```
\c company_prod
```

Create table :

```
CREATE TABLE hr.replication_test (
```

```
id SERIAL PRIMARY KEY,
message VARCHAR(100)
);
```

Insert data :

```
INSERT INTO hr.replication_test
(message)
VALUES
('Replication Operational');
```

```
company_prod=# CREATE TABLE hr.replication_test (
id SERIAL PRIMARY KEY,
message VARCHAR(100)
);
CREATE TABLE
company_prod=# INSERT INTO hr.replication_test
(message)
VALUES
('Replication Operational');
INSERT 0 1
company_prod=#
```

2. Verify on Replica

On pg-replica :

```
\c company_prod
```

Query :

```
SELECT *
FROM hr.replication_test;
```

```
postgres=# \c company_prod
You are now connected to database "company_prod" as user "postgres".
company_prod=# SELECT * FROM hr.replication_test
company_prod=# ^C
company_prod=# SELECT * FROM hr.replication_test;
 id |      message
-----+-----
  1 | Replication Operational
(1 row)
company_prod=#
```

3. Read-Only Validation

Attempt :

```
INSERT INTO hr.replication_test
(message)
VALUES
('Should Fail');
```

Expected :

```
ERROR:
```

cannot execute INSERT in a read-only transaction

```
company_prod=# INSERT INTO hr.replication_test
(message)
VALUES
('Should Fail');
ERROR:  cannot execute INSERT in a read-only transaction
company_prod=#
```

XVII. Replication Monitoring

1. Check Replica Status

On replica :

```
SELECT
status,
receive_start_lsn,
latest_end_lsn
FROM pg_stat_wal_receiver;
```

2. Check Replication Slots

On primary :

```
SELECT
slot_name,
active
FROM pg_replication_slots;
```

Expected :

```
replica_slot | t
You are now connected to database postgres as user postgres.
postgres=# SELECT
slot_name,
active
FROM pg_replication_slots;
slot_name | active
-----+-----
replica_slot | t
(1 row)
postgres=#
```

3. Check Replication Lag

On primary :

```
SELECT
client_addr,
state,
write_lag,
flush_lag,
replay_lag
```

```
FROM pg_stat_replication;
```

Expected :

Near zero lag

```
postgres=# SELECT
client_addr,
state,
write_lag,
flush_lag,
replay_lag
FROM pg_stat_replication;
 client_addr | state | write_lag | flush_lag | replay_lag
-----+-----+-----+-----+-----
192.168.10.21 | streaming |          |          |
(1 row)

postgres=#
```

XVIII. Simulated Failover Procedure

1. Simulate Primary Failure

On pg-primary :

```
sudo systemctl stop postgresql
```

2. Promote Replica

On pg-replica :

```
sudo -u postgres psql
```

Run :

```
SELECT pg_promote();
```

Expected :

t

```
postgres=# SELECT pg_promote();
 pg_promote
-----
t
(1 row)
```

Verify :

```
SELECT pg_is_in_recovery();
```

Expected :

f

```
postgres=# SELECT pg_is_in_recovery();
 pg_is_in_recovery
-----
f
(1 row)

postgres=#
```

Meaning : Replica became primary

XIX. Post-Failover Validation

Create est record :

```
INSERT INTO hr.replication_test
(message)
VALUES
('Failover Successful');
```

```
postgres=# \c company_prod
You are now connected to database "company_prod" as user "postgres".
company_prod=# INSERT INTO hr.replication_test
(message)
VALUES
('Failover Successful');
INSERT 0 1
company_prod=# SELECT * FROM hr.replication_test;
 id |      message
-----+-----
  1 | Replication Operational
 34 | Failover Successful
(2 rows)
company_prod=#
```

XX. High Availability Assessment

Current HA Level :

Feature	Status
Streaming Replication	✓
Physical Replica	✓
Replication Slot	✓
WAL Streaming	✓
Failover Procedure	✓
Monitoring	Partial
Automatic Failover	Not Yet

XXI. Enterprise Improvements (Future)

- **Load Balancing**
 - HAProxy
- **Virtual IP Failover**
 - Keepalived
- **Cluster Management**
 - Patroni
- **Connection Pooling**
 - PgBouncer